

Text Mining – Practical Session 2025

Cédric BOSCHER
cedric.boscher@univ-st-etienne.fr

November 7, 2025

Introduction

The Reuters Corpus is a large collection of news documents from the late 20th century, mainly used for research in natural language processing (NLP), text classification, and information retrieval. The documents are categorized into various topics like commodities, economics, and finance. These documents are annotated with multiple categories such as industry sectors and economic topics.

Documents with the category **earn** typically refer to news articles focusing on financial gains, stock market increases, or business profit reports.

In this practical session, we will put a focus on the Reuters Corpus, with the objective of classifying correctly the documents that belong to the **earn** category.

The goal is to propose approaches to automatically classify documents into two categories:

- **GAIN (1)** : documents that refer to earnings, i.e., that include the category label **earn**.
- **NO_GAIN (0)** : documents that do not include the category label **earn**.

During this session, your task will be to explore the corpus, pre-process the text data, and experiment two classification approaches, respectively based on TF-IDF matrices and Word Embeddings.

Work Submission

At the end of the practical session, you are expected to submit your code (either .py or .ipynb files) on moodle, in the Data Mining Course, under the **Text Mining - Practical Work Submission** form. The submission will grant you 1 point on your evaluation grade. Please submit your code in an archive file named after the following convention : **NAME_SURNAME.zip**.

Technical Environment

This Practical session will focus graph mining with Python. We will use the NetworkX and Scikit-Learn. The code will be written with Python 3.

As mentioned in the mail you received before the practical session, you have two options :

-
1. Use your personal computer , provided you can run Python with a virtual environment on it without issues. You can try creating a virtual environment by following the quick-start section of the documentation and testing the installation of a library (for instance: pip install gensim).
 2. Alternatively, you may use Google Colab to create a Python notebook in a remotely hosted virtual environment. Please note that this option requires a Google account (apologies for any inconvenience).

Option 1 - Virtual Environment on your personal machine machine

1. First, make sure that you have a correct version of Python installed.

In the console, type `python --version`.

2. Then, you can create a virtual environment with `venv`, for instance:

```
python -m venv <your_environment_name>
```

3. Then, activate your environment as follows:

```
source <your_environment_name>/bin/activate
```

4. Then install the libraries:

```
pip install networkx scikit-learn matplotlib node2vec
```

Option 2 - Use Google Colab

1. **Open Google Colab:**

- Visit <https://colab.research.google.com> and log in with your Google account.

2. **Start a New Notebook:**

- Click on the *"File"* menu and select *"New notebook"*.
- Alternatively, you can click *"New Notebook"* directly on the homepage.

3. **Rename Your Notebook:**

- Click on the default title (usually *"Untitled"*) at the top left and type your desired name.

4. **Add and Edit Cells:**

- Use the *" + Code"* or *" + Text"* buttons to add code and text cells as needed.
- Code cells run Python code, while text cells can contain formatted text using Markdown.

5. **Save and Access:**

- The notebook automatically saves to your Google Drive, so you can access it anytime in the *"Colab Notebooks"* folder.

1 Data Exploration

1. Using `nltk`, download the Reuters corpus by running:
`nltk.download('reuters')`
2. Import `reuters` from the `nltk.corpus` sub-package. Access the IDs of the documents with `reuters.fileids()`.
3. You can access the text of each document using `reuters.raw(document_id)`. Try printing the text of the first document in the corpus.
4. You can also access the categories of each document using `reuters.categories(document_id)`. Try printing the categories of the first document in the corpus.
5. Print the size of your corpus. You should obtain 10,788 documents.

For now, just consider the corpus as a whole and ignore the train/test labels in the documents' IDs. The train/test splits will be re-shuffled later in the next sections.

2 Data Preprocessing

During the lecture, you learned several steps to pre-process the data. You will apply several steps of pre-processing to the Reuters Corpus.

1. Create an array containing, for each document, a tuple or a dictionary containing its text and its category.
2. For each document, turn the text to lowercase.
3. Then, tokenize the text. You can find a Tokenizer with `from nltk import word_tokenize`.
4. Then, apply lemmatization to the text. You can find a Lemmatizer with `from nltk import WordNetLemmatizer`.
5. Remove Stop Words. You can find the stop words with `from nltk.corpus import stopwords`. First, try to print the list of English stop words.

3 Document Classification with Document-Term Matrices

Reminder: we want to automatically classify documents into two categories:

- **GAIN (1)** : documents that refer to earnings, i.e., that include the category label **earn**.
- **NO_GAIN (0)** : documents that do not include the category label **earn**.

In this section, you will learn how to classify documents by using the TF-IDF matrix of your corpus as classification features.

1. Make sure you have an array `texts` containing the pre-processed text for each document (such that `texts[0]` contains the text of the first document of the corpus). Then, define a second array `labels` that contains a class value for each document (1 or 0).

2. Split your dataset into a train and test set using the `train_test_split` method of `scikit-learn` (use `from sklearn.model_selection import train_test_split`). Split your data as follows:

```
X_train, X_test, y_train, y_test = \
train_test_split(texts, labels, test_size=0.2, random_state=42)
```

3. Compute the document-term matrix `X_matrix` from the training text corpus `X_train`.

You may use the `TfidfVectorizer` class from `scikit-learn`:

```
from sklearn.feature_extraction.text import TfidfVectorizer.
```

Note : If you want to display the matrix in a human-readable format, you can use the following function:

```
import pandas as pd
```

```
def print_matrix(X, vectorizer):
    dense_matrix = X.todense()
    feature_names = vectorizer.get_feature_names_out()
    df = pd.DataFrame(dense_matrix, columns=feature_names)
    print(df)
```

4. Train a Decision Tree Classifier by using the `X_train` matrix as classification features, and the `y_train` array containing the class for each document. You might use the `DecisionTreeClassifier` class from `scikit-learn` with default settings, as follows :

```
from sklearn.tree import DecisionTreeClassifier
```

```
clf = DecisionTreeClassifier()
clf.fit(X_matrix, y_train)
```

5. Display the 10 most discriminant features from your trained Decision Tree, i.e., the 10 terms that discriminate the most documents belonging to the **GAIN** class.

(hint: you might use the `feature_importance_` attribute of the `DecisionTreeClassifier` instance to show the discriminant importance of each feature, and the `get_feature_names_out()` method of your `TfidfVectorizer` instance to map feature ids with corresponding words. You may proceed as follows :

```

import numpy as np
# Get the feature importances
importances = clf.feature_importances_

# Get the feature (term) names
feature_names = vectorizer.get_feature_names_out()

# Sort the features by importance
indices = np.argsort(importances)[::-1]

# Print the top n most discriminant terms (e.g., top 10)
top_n = 10
for i in range(top_n):
    print(f"{i+1}. Feature: {feature_names[indices[i]]}, \
    Importance: {importances[indices[i]]}")

```

6. Evaluate your Decision Tree on your test corpus, by computing its Accuracy (Hint: for the test set, use the `transform` method of your vectorizer instead of `fit_transform`, as follows).

```

from sklearn.metrics import accuracy_score

X_matrix = vectorizer.transform(X_test)
y_pred = clf.predict(X_matrix)
accuracy = accuracy_score(y_test, y_pred)

```

4 Document Classification with Word Embeddings

One of the most famous Word Embedding representations is Word2Vec. It aims to compute a low-dimensional representation of the words of a corpus. In this section, you will classify documents by using Word2Vec and Doc2Vec representations, which differ from the TF-IDF features-based approach.

1. First, import `gensim` and load the pre-trained model `glove-wiki-gigaword-50` as follows:

```
model = gensim.downloader.load("glove-wiki-gigaword-50")
```

2. Word2Vec transforms words into an embedded vector representation, allowing the modeling and verification of similarity between semantically close terms.

Try to compute the embedding of the term "profit" as follows:

```
embedding = model['profit']
```

3. Find the 10 most similar terms with:

```
sims = model.most_similar('profit', top_n=10)
```

-
4. Now, you will implement a naive Doc2Vec approach: write a function that computes the average embedding of a document, i.e., the average of terms' embeddings, for all terms in the document that are found in the pre-trained model's vocabulary.
 5. With the function you implemented, compute the Doc2Vec embeddings of the documents from your training and test corpus.
 6. Then, try training and evaluating several `sklearn` classifiers such as a `LogisticRegression` or `svm.SVC` on your classification task, using the produced embeddings as training features.
 7. Compare your results with the results you obtained in Section 3. In that case, which solution is the most performant one ? And the most interpretable one ?